

TALLER DE MIGRACIÓN: REEMPLAZO DE BACKEND

Objetivo: Realizar la **migración completa del backend** de un sistema REST. Los estudiantes tomarán el **mismo tema libre** (Mascotas, Libros, Películas, etc.) que desarrollaron en el taller anterior de Java Spring Boot, y **reemplazarán por completo ese backend de Java por uno desarrollado en Node.js (Express + TypeScript + Prisma)**. Se mantendrá el frontend de **React** intacto (solo apuntando al nuevo servidor) y la base de datos de PostgreSQL para demostrar el concepto de interoperabilidad y migración de servicios.

💡 Concepto de la Migración

En el desarrollo de software real, es común migrar backends entre tecnologías por razones de rendimiento, costos o mantenimiento. En este taller aprenderás que:

1. **El Frontend es agnóstico:** A tu aplicación de React no le interesa en qué lenguaje está escrito el servidor (Java o Node), siempre y cuando los endpoints reciban y devuelvan exactamente el mismo formato de datos y manejen los mismos tokens JWT.
2. **La Base de Datos se mantiene:** Seguiremos usando tu misma base de datos relacional de PostgreSQL con su información e imágenes binarias intactas.

📁 ESTRUCTURA DE CARPETAS DEL PROYECTO (MIGRADO A NODE.JS)

Los estudiantes estructurarán su nuevo backend en Node.js siguiendo la arquitectura de capas limpia:

```
└─ taller_migracion_backend/
   └─ prisma/
      └─ schema.prisma (Definición de modelos de base de datos extraídos o creados)
   └─ src/
      └─ database/
         └─ prisma.ts (Conexión única con Pool de pg adaptado a Prisma)
      └─ middlewares/
         └─ auth.middleware.ts (Verificación de tokens JWT)
      └─ controllers/
         └─ item.controller.ts (Lógica CRUD de tu mismo tema de Java)
      └─ routes/
         └─ items.ts (Definición de endpoints mapeados exactamente igual a Java)
      └─ index.ts (Servidor Express principal)
```

PASO A PASO DEL TALLER

Paso 1: Configurar la Base de Datos Existente con Prisma 7

1. Inicializa Prisma en tu nuevo proyecto de Node.js.
2. Apunta la variable `DATABASE_URL` en tu archivo `.env` a la **misma base de datos de PostgreSQL** que usaste en tu taller de Java.
3. Utiliza el comando de introspección para no tener que crear las tablas de nuevo:
- 4.

```
❏ npx prisma db pull
```

5. ☐ Esto leerá tus tablas de Java (por ejemplo, *Mascotas o Películas*) y creará automáticamente los modelos en tu `schema.prisma`.
6. Genera el cliente de Prisma:
- 7.

```
❏ npx prisma generate
```

```
❏
```

Paso 2: Reescribir los Endpoints de Java a Node.js (CRUD)

En tu controlador (`src/controllers/item.controller.ts`), debes reescribir la lógica que tenías en Spring Boot (`ItemController.java`) a código TypeScript usando Prisma Client:

1. Endpoint de Eliminación (DELETE `/auth/items/:id`)

- **Lógica:** Buscar por ID en Prisma y eliminar el registro. Retornar código `200 OK` con una confirmación JSON si fue exitoso (igual a como respondía Java).

2. Endpoint de Edición (PUT `/auth/items/:id`)

- **Lógica:** Recibe el ID por parámetro y los textos en el cuerpo (`req.body`).
- **Opcionalidad del Archivo (Multer):** Si el cliente no sube una foto nueva (`req.file` es `undefined`), debes conservar los bytes de la foto existente consultando el registro previamente en PostgreSQL. Si sube una foto nueva, actualiza el buffer de la foto (`Buffer.from(req.file.buffer)`) y su mimetype correspondientes.

Paso 3: Conectar el Frontend en React al Nuevo Backend

1. Abre el código de tu frontend en React (`mi-proyecto-jwt`).
2. Ve al archivo de configuración de endpoints (`src/config/apiConfig.js`) y cambia la URL de conexión del puerto `8080` (Java) al puerto `3000` (Node.js).
3. Ejecuta tu React localmente y realiza pruebas. Deberías poder:
 - Listar las tarjetas (Cards) con las fotos recuperadas del nuevo backend de Node.
 - Eliminar un elemento desde la Card (llamando al `DELETE` de Node).
 - Editar un elemento mediante el modal (llamando al `PUT` de Node).

ENTREGABLES REQUERIDOS (2 CAPTURAS DE PANTALLA)

Deberás presentar tu reporte con las siguientes evidencias a pantalla completa:

1. **Captura 1 (Interoperabilidad):** Vista en el navegador de tu aplicación React cargando y pintando perfectamente tus tarjetas con sus imágenes, consumidas ahora desde el backend de Node.js.
2. **Captura 2 (Flujo de Eliminación - Network):** Inspector de red abierto en el navegador mostrando el estado **200 OK** tras hacer la petición **DELETE** a tu API de Node.js al eliminar una Card.

. Entrega

- Link del Repositorio
- Capturas de Pantalla Enviadas al grupo de WhatsApp:

